ARTIFICIAL INTELLIGENCE

Practical Notes - TYBSc (C.S.) Semester V

Each practical: the idea, the shortest working code, and its output.

Contents

1 Breadth-First Search & Iterative Deepening DFS

2-A A* Search

2-B Recursive Best-First Search (RBFS)

3 Decision Tree Learning

4 Feed Forward Backpropagation Neural Network

5 Support Vector Machine (SVM)

6 AdaBoost Ensemble Learning

7 Naive Bayes Classifier

8 K-Nearest Neighbors (K-NN)

9 Association Rule Mining - Apriori

10 TensorFlow Tools and Libraries

1 Breadth-First Search & Iterative Deepening DFS

Idea BFS explores the graph level by level using a queue (FIFO), so it always finds the shortest path when every edge costs the same. IDDFS runs depth-limited DFS again and again with limits 0, 1, 2, ... until the goal appears. It matches BFS in completeness but keeps DFS memory; compare them on path length, nodes expanded and time.

Code

```
from collections import deque

G = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'],
     'E': ['G', 'H'], 'F': ['I']}             # I is the goal

def bfs(start, goal):
    q, seen = deque([[start]]), set()
    while q:
        path = q.popleft()                    # oldest path first (FIFO)
        node = path[-1]
        if node == goal:
            return path
        if node not in seen:
            seen.add(node)
            for nxt in G.get(node, []):
                q.append(path + [nxt])

def iddfs(start, goal, max_depth=5):
    def dls(node, path, depth):               # depth-limited DFS
        if node == goal:
            return path
        if depth == 0:
            return None
        for nxt in G.get(node, []):
            if nxt not in path:
                found = dls(nxt, path + [nxt], depth - 1)
                if found:
                    return found
    for limit in range(max_depth + 1):        # try limits 0, 1, 2, ...
        found = dls(start, [start], limit)
        if found:
            return found

print("BFS  :", bfs('A', 'I'))
print("IDDFS:", iddfs('A', 'I'))
```

Output

```
BFS  : ['A', 'C', 'F', 'I']
IDDFS: ['A', 'C', 'F', 'I']
```

2-A A* Search

Idea A* is informed search on a weighted graph. It always expands the node with the smallest f(n) = g(n) + h(n), where g is the cost paid so far and h is the heuristic estimate of what is left. If h never overestimates (admissible), the returned path is guaranteed to be the cheapest one.

Code

```
import heapq

G = {'A': [('B', 4), ('C', 5)], 'B': [('D', 7)],
     'C': [('D', 2)], 'D': [('E', 9)], 'E': []}
H = {'A': 7, 'B': 6, 'C': 2, 'D': 1, 'E': 0}   # admissible heuristic

def a_star(start, goal):
    open_list = [(H[start], start)]            # (f, node)
    g = {start: 0}
    parent = {start: None}
    while open_list:
        _, node = heapq.heappop(open_list)     # smallest f first
        if node == goal:
            path = []
            while node:                        # walk parents back
                path.append(node)
                node = parent[node]
            return path[::-1], g[goal]
        for nxt, cost in G[node]:
            new_g = g[node] + cost
            if nxt not in g or new_g < g[nxt]:
                g[nxt] = new_g
                parent[nxt] = node
                heapq.heappush(open_list, (new_g + H[nxt], nxt))

path, cost = a_star('A', 'E')
print("Path:", ' -> '.join(path), "| Cost:", cost)
```

Output

```
Path: A -> C -> D -> E | Cost: 16
```

2-B Recursive Best-First Search (RBFS)

Idea RBFS is best-first search compressed into linear memory. It expands recursively like DFS while carrying an f-limit: the f-value of the best forgotten sibling. When every child's f exceeds the limit, it backtracks and tries the alternative branch instead.

Code

```
G = {'A': [('B', 1), ('C', 4)], 'B': [('C', 2), ('D', 5)],
     'C': [('D', 1)], 'D': [('E', 3)], 'E': []}
H = {'A': 7, 'B': 6, 'C': 2, 'D': 1, 'E': 0}

def rbfs(node, g, f_limit, path):
    if node == 'E':                            # goal reached
        return path, g
    succ = [[nxt, g + c, max(g + c + H[nxt], H[node])]
            for nxt, c in G[node] if nxt not in path]
    if not succ:
        return None, float('inf')
    while True:
        succ.sort(key=lambda s: s[2])          # best f first
        best = succ[0]
        if best[2] > f_limit:                  # worse than sibling branch
```

```
            return None, best[2]              # backtrack
        alt = succ[1][2] if len(succ) > 1 else float('inf')
        result, f = rbfs(best[0], best[1],
                         min(f_limit, alt), path + [best[0]])
        best[2] = f                           # update with real f
        if result:
            return result, f

path, cost = rbfs('A', 0, float('inf'), ['A'])
print("Path:", ' -> '.join(path), "| Cost:", cost)
```

Output

```
Path: A -> B -> C -> D -> E | Cost: 7
```

3 Decision Tree Learning

Idea A decision tree splits the data repeatedly on the feature that best purifies the groups (Gini impurity or information gain) until the leaves are mostly single-class. Prediction simply walks the tree as if-else rules, which makes the model highly readable. Test accuracy and the confusion matrix show how well it generalizes to unseen data.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
Xtr, Xte, ytr, yte = train_test_split(iris.data, iris.target,
                                      test_size=0.25, random_state=42)
tree = DecisionTreeClassifier(random_state=42).fit(Xtr, ytr)  # Gini
pred = tree.predict(Xte)

print("Accuracy:", accuracy_score(yte, pred))
print(confusion_matrix(yte, pred))
print(export_text(tree, feature_names=iris.feature_names))
```

Output

```
Accuracy: 1.0
[[15  0  0]
 [ 0 11  0]
 [ 0  0 12]]
|--- petal length (cm) <= 2.45
|   |--- class: 0
... (the full if-else tree continues)
```

4 Feed Forward Backpropagation Neural Network

Idea Inputs flow forward through a hidden layer to produce predictions; the output error is then propagated backwards and every weight is nudged downhill by gradient descent (chain rule). Scaling the features first helps convergence. The cycle repeats over epochs until the accuracy stops improving.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
```

```
X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=42)
sc = StandardScaler()                          # scale features first
Xtr, Xte = sc.fit_transform(Xtr), sc.transform(Xte)

net = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000,
                    random_state=42).fit(Xtr, ytr)
print("Actual   :", yte)
print("Predicted:", net.predict(Xte))
print("Accuracy :", accuracy_score(yte, net.predict(Xte)))
```

Output

```
Actual   : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 ...]
Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 ...]
Accuracy : 1.0
```

5 Support Vector Machine (SVM)

Idea SVM finds the hyperplane that separates two classes with the largest margin; the closest points, the support vectors, define that hyperplane. C sets the penalty for margin violations, and kernels (linear, RBF) handle non-linear boundaries. The model is judged on held-out test data.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = load_iris(return_X_y=True)
X, y = X[:100], y[:100]                        # two classes only
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                      random_state=4)
svm = SVC(kernel='linear', C=1).fit(Xtr, ytr)  # C = margin penalty
pred = svm.predict(Xte)

print("Accuracy:", accuracy_score(yte, pred))
print(confusion_matrix(yte, pred))
```

Output

```
Accuracy: 1.0
[[14  0]
 [ 0  6]]
```

5-B of the journal only adds a matplotlib decision-boundary plot on top of this code.

6 AdaBoost Ensemble Learning

Idea AdaBoost trains weak learners (depth-1 decision stumps) one after another. After each round it raises the weights of the misclassified samples, so the next stump concentrates on them. The final prediction is a weighted vote of all stumps, which beats any single stump.

Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      random_state=4)
stump = DecisionTreeClassifier(max_depth=1,
                               random_state=4).fit(Xtr, ytr)
ada = AdaBoostClassifier(n_estimators=50,
                         random_state=4).fit(Xtr, ytr)

print("Weak stump accuracy:", accuracy_score(yte, stump.predict(Xte)))
print("AdaBoost accuracy  :", accuracy_score(yte, ada.predict(Xte)))
```

Output

```
Weak stump accuracy: 0.8888888888888888
AdaBoost accuracy  : 0.9649122807017544
```

7 Naive Bayes Classifier

Idea Naive Bayes applies Bayes' theorem with the naive assumption that features are independent given the class. Training only counts prior probabilities and per-feature likelihoods, and prediction picks the class with the highest posterior. It is fast and surprisingly accurate on small data.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                      random_state=42)
nb = GaussianNB().fit(Xtr, ytr)
pred = nb.predict(Xte)

print("Accuracy:", accuracy_score(yte, pred) * 100, "%")
print("Probabilities (first 3 rows):")
print(nb.predict_proba(Xte)[:3].round(3))
print(confusion_matrix(yte, pred))
```

Output

```
Accuracy: 100.0 %
Probabilities (first 3 rows):
[[0.    0.996 0.004]
 [1.    0.    0.   ]
 [0.    0.    1.   ]]
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

8 K-Nearest Neighbors (K-NN)

Idea K-NN has no real training phase: it stores the data and, for a new point, takes a majority vote among its k nearest neighbors (Euclidean distance). Here k = 3 and only the two sepal features are used. Accuracy depends heavily on the choice of k and on feature scale.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data[:, :2]                  # sepal length + width only
Xtr, Xte, ytr, yte = train_test_split(X, iris.target, test_size=0.3,
                                      random_state=43)
knn = KNeighborsClassifier(n_neighbors=3).fit(Xtr, ytr)
print("Accuracy:", accuracy_score(yte, knn.predict(Xte)) * 100, "%")

new = [[5.0, 3.5]]                    # an unseen flower
p = knn.predict(new)[0]
print("New data:", new, "->", iris.target_names[p])
```

Output

```
Accuracy: 80.0 %
New data: [[5.0, 3.5]] -> setosa
```

9 Association Rule Mining - Apriori

Idea Apriori finds itemsets that appear in at least min_support of the transactions, using the property that every subset of a frequent itemset must also be frequent. Support measures how often an itemset appears; confidence and lift then grade the rules such as Bread -> Butter.

Code

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori

T = [['Milk', 'Bread'], ['Bread', 'Butter'],
     ['Milk', 'Bread', 'Butter'], ['Milk', 'Butter'],
     ['Bread', 'Butter']]

te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(T), columns=te.columns_)
print(df.astype(int))

items = apriori(df, min_support=0.4, use_colnames=True)
print(items.to_string(index=False))
```

Output

```
 support                itemsets
     0.8         frozenset({Bread})
     0.8        frozenset({Butter})
     0.6          frozenset({Milk})
     0.6 frozenset({Butter, Bread})
     0.4   frozenset({Milk, Bread})
     0.4  frozenset({Butter, Milk})
```

10 TensorFlow Tools and Libraries

Idea TensorFlow + Keras builds a network layer by layer, then follows the compile (optimizer + loss) -> fit (train) -> evaluate (test) workflow. Fashion-MNIST holds 28x28 grayscale images of 10 clothing classes, scaled to 0-1 before training. The softmax output gives one probability per class.

Code

```
import tensorflow as tf
from tensorflow.keras import layers, models

(Xtr, ytr), (Xte, yte) = tf.keras.datasets.fashion_mnist.load_data()
Xtr, Xte = Xtr / 255.0, Xte / 255.0   # scale pixels to 0-1

model = models.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),                 # 784 inputs
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')   # 10 clothing classes
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(Xtr, ytr, epochs=3, batch_size=64, validation_split=0.1)
loss, acc = model.evaluate(Xte, yte)
print("Test accuracy:", acc)
```

Output

```
Epoch 3/3  accuracy: 0.87  val_accuracy: 0.88
Test accuracy: ~0.86   (varies slightly per run)
```

Shortened to a dense network; the journal's CNN simply swaps the Flatten layer for two Conv2D + MaxPooling blocks.